Understanding the CheckList Paper for the Presentation

> **AI-generated content (Claude).** Written from a full reading of the paper. Review before presenting.

Paper: Ribeiro, M. T.; Wu, T.; Guestrin, C.; Singh, S. *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*. ACL 2020, pp. 4902–4912. **PDF:** https://aclanthology.org/2020.acl-main.442.pdf (11 pages) **Slides:** “CheckList Seminar - Slides (final)” in Google Drive (13 slides)

The idea in 30 seconds

Language models are usually judged by **one number**: their accuracy on a test set. A model with 92% looks great. The paper shows that this number hides serious failures: the same model can get simple sentences like *“I thought the plane would be awful, but it wasn’t”* wrong most of the time.

The proposal, called **CheckList**, is to test language models **the way we test software**: one skill at a time, with many test cases generated automatically.

The sentence to remember: **CheckList changes the question from “How accurate is the model?” to “What can the model actually do?”**

Slide map

Part	Slides	Topic
—	1	Cover
Part 1	2 · 3 · 4	Motivation · What the paper contributes · Related work
Part 2	5 · 6 · 7	Accuracy vs. behavioral testing · Capabilities, perturbations, metamorphic tests · Tasks and models
Part 3	8 · 9	The matrix · The three test types
Part 4	10 · 11 · 12	Primary results · Applications of CheckList · Conclusion
—	13	References

Part 1 — Motivation, contributions and related work

The problem behind the paper (slide 2)

The standard way to evaluate an NLP (Natural Language Processing) model is:

1. split the data into training, validation and test sets;
2. train the model;
3. measure accuracy on the test set, often on a public leaderboard.

The authors point out three weaknesses:

- **Biased test sets:** the test set has the same biases as the training data, so real-world performance can be much lower than reported.
- **One number:** a single score doesn't show *where* the model fails or how to fix it.
- **Narrow alternatives:** other evaluations existed (robustness, fairness…), but each looked at one task or one skill. There was no general guide on what to test.

The example on the slide: BERT reaches **92.7%** accuracy on the SST-2 sentiment benchmark, yet it fails **85%** of simple sentences with the negation at the end, like *"I thought the plane would be awful, but it wasn't"*. The sentence is positive, because the plane was *not* awful. The authors' answer: test language models the way we test software.

How to close the slide: "So, if accuracy isn't enough, what does the paper propose?"

What the paper contributes (slide 3)

The slide sums up the paper in **three contributions**:

#	Contribution	In plain words
1	**A method**	Test language models like software: **one skill at a time** (negation, names…) instead of one overall score. It works for any task and any model.
2	**A free tool**	Generates **thousands of tests automatically** from sentence templates. Available at github.com/marcotcr/checklist.
3	**Evidence**	Found serious bugs in research models (BERT) and in products from Microsoft, Google and Amazon. People using CheckList found **about 3× more bugs**.

The details come later: the list of skills and the test types in Part 3, the bug numbers in Part 4.

One-line summary: "Three things: a method, a tool, and evidence that it works."

Related work (slide 4)

The slide title is **“Before CheckList: one piece at a time”**. The message: **each earlier approach solved only one piece of the problem.**

Card	What it was	Why it wasn’t enough
Accuracy	The overall score on a benchmark	**Only one number**: it doesn’t show where the model fails
Robustness and fairness tests	Tests with typos, swapped names…	Look at **one skill or one task** at a time
Challenge sets	Sets of difficult sentences	**Small and hand-made**
Knowledge tests (called *probes* in the paper)	Small classifiers that read the model’s internals to see what it learned	Show the model **knows** something, not that it **uses** it on the real task

The bottom line: CheckList brings all of these together in one method, using ideas from **software testing** (unit tests and metamorphic tests).

Good example for knowledge tests: a knowledge test shows BERT recognizes names with 96.7% accuracy, but BERT fine-tuned on a real task still fails simple tests with names. Knowing something inside doesn’t mean the model uses it.

What CheckList doesn’t cover, according to the authors: labelling errors, data versioning, security attacks and interpretability. It’s not on the slide, but useful if someone asks.

Part 2 — Key concepts

Accuracy vs. behavioral testing (slide 5)

- **Held-out accuracy:** the percentage of correct answers on examples the model never saw during training. Example: BERT reaches 92.7% on SST-2 (movie reviews).
- **Behavioral (black-box) testing:** testing only through inputs and outputs, without looking at code or weights. Each test checks one behavior.

Analogy: accuracy is a student’s **final average**; behavioral testing is the **report card per subject**. A student with a great average can still be failing one subject completely.

Capabilities, perturbations and metamorphic tests (slide 6)

Concept	What it is	Example
Capability	A skill the task requires	Negation: “not poor” is not negative
Perturbation	A small, controlled change to the input	Replacing “Chicago” with “Dallas” in a tweet
Metamorphic test	Compares outputs with each other, with no answer key	If I only changed the city, the sentiment must stay the same

Why metamorphic tests matter: I don't know the "correct" sentiment of a random tweet, but I know that changing the city name shouldn't change it. So we can test on real data **without labels**.

Analogy: checking a bathroom scale. Without knowing your exact weight, you know that taking off your watch can't make the number go up.

Tasks and models (slide 7)

Task	What the model must do
Sentiment analysis	Say whether a text is positive, negative or neutral
Quora Question Pairs (QQP)	Say whether two questions ask the same thing
Reading comprehension (SQuAD)	Answer a question from a paragraph

- **BERT and RoBERTa:** Transformer language models, pretrained on large amounts of text and then fine-tuned for each task.
- **Masked language model:** predicts a hidden word in a sentence. Given "I really [mask] the flight", it suggests *enjoyed*, *liked*, *hated*... CheckList uses this to help generate tests.

Part 3 — Methodology

The matrix (slide 8)

Testing is organized as a table:

- **rows** = capabilities (vocabulary, named entities, negation...);
- **columns** = test types (MFT, INV, DIR);
- **each cell** = the failure rate, i.e. the percentage of cases the model got wrong.

Example from the paper (Figure 1), with a commercial sentiment model:

Capability	MFT	INV	DIR
Vocabulary	15.0%	16.2%	34.6%
Named entities	0.0%	20.8%	—
Negation	**76.4%**	—	—

In other words, the model fails 76.4% of a simple negation test. Because this is black-box testing, no access to the model's internals is needed, which is why the authors could test paid commercial APIs.

The three test types (slide 9)

Type	Idea	Example	Expected behavior
MFT – Minimum Functionality Test	Simple cases with a known answer (like a unit test)	“I didn’t love the flight.”	Negative
INV – Invariance test	A change that **should not** matter	Replace “Cuba” with “Canada”	Same prediction
DIR – Directional Expectation test	A change with a known effect	Append “You are lame.”	Sentiment must not become more positive

Key point: INV and DIR **need no labels**. They compare the answer before and after the change.

Everyday example (an app that reads restaurant reviews):

- **MFT:** “The food was not bad” must come out positive or neutral, never negative.
- **INV:** “restaurant in Boston” → “restaurant in Denver” must not change the score.
- **DIR:** appending “Terrible service” must not make the review better.

Background: how tests are generated and how the study was run

This content is no longer on its own slide, but it helps to know it if someone asks.

Templates. Writing thousands of tests by hand is impossible, so the tool uses sentence templates:

```
I {NEGATION} {POS_VERB} the {THING}.
{NEGATION} = didn't, can't say I, ...
{POS_VERB} = love, like, ...
{THING}    = food, flight, service, ...
```

Every combination becomes a test case. The tool also suggests words with RoBERTa, provides lists of names and places, and includes ready-made perturbations such as typos.

The study had two parts:

1. **Testing the models:** three tasks, the commercial APIs of Microsoft, Google and Amazon, plus BERT and RoBERTa. A case fails when the label is wrong, or, for INV and DIR, when the prediction moves the wrong way by more than 0.1 in probability.
2. **Testing with people:**
    - a 5-hour session with Microsoft’s team that maintains a commercial sentiment model;
    - a controlled experiment with 18 people (8 from industry, 10 from academia), 2 hours each, in three groups of 6: no help, a list of capabilities, or the full tool.

Part 4 — Results, applications and conclusion

Primary results (slide 10)

The slide has two columns: on the left, the models' failures; on the right, the results with users.

Models fail the basics (left column)

Number	What happened
100%	Negation at the end ("…would be awful, but it wasn't"): Microsoft and Amazon fail every case
100%	Reading comprehension: fails every simple "he"/"she" test ("He is a journalist, and she is an adviser. Who is a journalist?")
98%	Quora pairs: BERT confuses "before" and "after"
89%	"John is not a doctor, Mary is. Who is a doctor?" → the model answers John. With the names swapped, it fails only 3.2%: a **gender bias**

Other important findings:

- **Research models beat commercial ones** on most tests, which surprised the authors. The exception is neutral sentences: BERT and RoBERTa fail almost all of them because they were trained only on positive and negative.
- **Accuracy hid the problem:** BERT is over 90% accurate on the negation sentences of SST-2, yet fails CheckList's simple negation tests.
- **The authors' interpretation:** models rely on **shortcuts** (for example, leaning on names) instead of real understanding.

CheckList users found more bugs (right column)

Comparing the group with the full tool and the group without help:

Measure	No help	With CheckList	Difference
Tests created	5.8	13.5	**2×**
Cases per test	7.3	198	**27×**
Serious bugs (severity ≥ 3)	2.2	6.2	**≈3×**

- An outside person re-rated bug severity and got nearly the same results.
- **Microsoft's team** found new bugs in a mature model, tested things they hadn't thought of (hashtags like "#IHateYou", implicit negation like "I wish it was good"), and said they would adopt CheckList.
- **Caveat:** each group had only 6 people and the paper reports no statistical test. It's a strong indication, not proof.

Applications of CheckList (slide 11)

The slide shows **where CheckList can be used in practice**, according to the paper:

Application	What it means	Where it comes from in the paper
Choosing a model	Compare models skill by skill, not by one score	The authors compared research models with Microsoft, Google and Amazon
Development cycle	Re-run the same tests on every new version of a model	Microsoft's team said they would add CheckList to their development cycle
Benchmarks and pipelines	Add the tests to existing benchmarks and evaluations	The paper's conclusion
Shared test suites	Teams build and reuse tests together	The authors expect collaborative test creation, since many tests work across tasks

The line at the bottom: all of this works on any model, even a paid API, because CheckList is black-box: it needs no access to training data or weights.

Conclusion (slide 12)

- **Accuracy is useful, but not enough.**
- **Strengths:** CheckList works for any task and model, finds bugs even in commercial products, and is easy to learn.
- **Limitations:**
  - it only tests behavior (it doesn't catch labelling errors or security problems);
  - **passing a test doesn't prove the model is correct** (commercial models passed the simple fairness test, which doesn't prove they are fair);
  - the user study is small (our observation, not the authors').
- **Future work proposed by the authors:** build test suites collaboratively, since many tests work across tasks.

Glossary

Term	Meaning
MFT	Minimum Functionality Test: simple cases with a known answer
INV	Invariance test: a change that should not alter the answer
DIR	Directional Expectation test: a change with a known effect on the answer
NER	Named Entity Recognition: handling names of people, places, companies
SRL	Semantic Role Labeling: understanding who did what to whom
POS	Part of Speech: word class (noun, verb, adjective…)
SST-2	Stanford Sentiment Treebank: movie-review sentiment benchmark
QQP	Quora Question Pairs: are two questions duplicates?
SQuAD	Stanford Question Answering Dataset: reading comprehension
Probe / knowledge test	A small classifier that reads a model's internal representations

---

Analogies for the presentation

- **Report card:** accuracy is the average; CheckList is the grade per subject.
- **Car inspection:** nobody approves a car just because it drove 100 km; there is a checklist (brakes, lights, tyres). The matrix is that checklist for language models.
- **The student who memorized keywords:** gets the usual exam right, fails when one detail changes ("before" vs. "after"). That is what the authors call a shortcut.
- **Bathroom scale:** without knowing your exact weight, you know taking off your watch can't increase the number. That is how INV and DIR work without an answer key.

Likely questions (short answers)

1. **What's wrong with accuracy?** It overestimates real performance, carries the biases of the training data, and doesn't show where the model fails.
2. **What is behavioral testing?** Testing only through inputs and outputs, without looking inside the model. That's why commercial APIs can be tested.
3. **What's the difference between MFT, INV and DIR?** MFT: simple case with a known answer. INV: a change that must not alter the answer. DIR: a change with a known effect on the answer.

4. **How can you test without labels?** INV and DIR compare the model's answer with its own previous answer, not with an answer key.
5. **Which failures were the most severe?** Negation (close to 100%), coreference (100%), time ("before"/"after") and semantic roles (who did what).
6. **Why did research models beat commercial ones?** The paper reports it as a surprise and gives no definitive explanation.
7. **Are the commercial models fair, since they passed the fairness test?** That can't be concluded. Passing a simple test doesn't prove the absence of bias.
8. **Isn't bug severity subjective?** Yes. To reduce that, an outside person re-rated it and got nearly the same result. With 6 people per group, it's an indication, not proof.
9. **How is CheckList different from knowledge tests (probes)?** Probes look inside the model; CheckList tests behavior on the real task.
10. **What are the limitations?** It doesn't cover labelling errors, data versioning, attacks or interpretability, and it depends on the creativity of whoever writes the tests.
11. **Where can CheckList be used in practice?** To choose between models, to re-test every new version, to extend benchmarks and evaluation pipelines, and to build shared test suites.

Check in the PDF before presenting

- The paper won the ACL 2020 Best Paper Award. This is not stated in the PDF; confirm before citing it.
- Which model always predicted negative in the fairness test for groups such as *black* and *gay*. The model symbol was lost in text extraction; from the paragraph order, it is one of the research models.